

Single-Agent versus Multi-Agent LLM Systems for Automated Programming: A Controlled Experiment in Software Engineering

LIANG YU*, Blekinge Institute of Technology, Sweden

JON PÅHLS, Ericsson, Sweden

EMIL ALÉGROTH, Blekinge Institute of Technology, Sweden

Large Language Models (LLMs) can produce working code, but complex tasks still demand decomposition, planning, and self-checking. Multi-agent systems address this by assigning role-specific prompts and using coordinators to route messages and invoke tools. Evidence from comparisons against single-agent setups is limited. We ran a controlled experiment on 133 LeetCode problems to test whether a multi-agent system beats a single agent on code quality. Agents were a Problem Analyzer, Solution Designer, Code Executor, and Solution Verifier. We also compared both LLM systems with the LeetCode human baseline. Metrics were acceptance rate, cyclomatic complexity, lines of code, and generation time. We replicated the experiment three times. Average acceptance rate was 98.50% (single-agent) and 97.74% (multi-agent), and both exceeded the LeetCode human baseline of 55.99%. The multi-agent system reduced mean cyclomatic complexity by 4.8% and lines of code by 3.6%. These gains came with a fourfold increase in end-to-end generation time. Quality gains from multiple agents were small and did not offset added cost (longer generation time, higher token usage, more API calls). For LeetCode-style coding tasks where fast turnaround matters, we recommend defaulting to a single agent and adopting multiple agents when specific quality targets (e.g., accuracy) justify the cost.

CCS Concepts: • **Software and its engineering** → **Automatic programming**; • **Computing methodologies** → **Artificial intelligence**.

Additional Key Words and Phrases: Large Language Models, Multi-Agent Systems, Code Generation, Empirical Software Engineering, Automated Programming

ACM Reference Format:

Liang Yu, Jon Pähls, and Emil Alégroth. 2025. Single-Agent versus Multi-Agent LLM Systems for Automated Programming: A Controlled Experiment in Software Engineering. *ACM Trans. Softw. Eng. Methodol.* 1, 1 (October 2025), 20 pages. <https://doi.org/XXXXXXX.XXXXXX>

1 Introduction

Large Language Models (LLMs) can generate functioning code [4] for many tasks. AI assistants for automated programming, such as GitHub Copilot [6], Amazon Q Developer¹, and Codeium², have brought the code generation capability into software development companies. Organizations, therefore, face a design question: how should these AI assistants be adopted and used to ensure that the generated outputs meet user expectations for higher-quality outcomes?

¹<https://aws.amazon.com/q/developer>

²<https://windurf.com>

Authors' Contact Information: Liang Yu, liang.yu@bth.se, Blekinge Institute of Technology, Karlskrona, Blekinge, Sweden; Jon Pähls, Ericsson, Karlskrona, Sweden, jon.pahls@ericsson.com; Emil Alégroth, Blekinge Institute of Technology, Karlskrona, Blekinge, Sweden, emil.alegroth@bth.se.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

Manuscript submitted to ACM

To address this design question, existing studies, such as [4, 12, 14], have investigated direct single-agent prompting, where one LLM produces executable scripts/functions/files (e.g., coding tasks [4]), and multi-agent solutions [12] that coordinate specialized roles such as planners, implementers, and verifiers (e.g., CAMEL [11] and MetaGPT [7]). Single- and multi-agent orchestrations can affect user acceptance and the quality of AI-generated outputs across software development tasks, and poor orchestration may reduce the performance of AI coding tools [13]. Yet comparing the performance of single and multi-agent designs remains challenging [8], leaving practitioners without clear decision support when choosing between these designs.

Despite recent studies on multi-agent LLMs [9, 16], industrial evidence and controlled tests that isolate agent orchestrations remain limited [7]. As a result, organizations may make design decisions based on intuition, which could lead to missed opportunities for improvement. We therefore examine whether the code-quality gains from multi-agent orchestration justify its added orchestration cost (e.g., end-to-end generation time, token usage, and API calls), and how the failure patterns differ between orchestrations.

In this study, we conducted a controlled experiment [15] with a fixed LLM to compare single-agent and multi-agent systems on coding problems. The coding problems target core programming skills in software development companies, including data structures, algorithmic trade-offs, and edge-case handling. Given this, we chose LeetCode³, which provides standardized programming problems suitable for controlled evaluation. Unlike many multi-agent evaluations that lack an external human benchmark, we include the LeetCode human acceptance rate to align our results with industrial practices.

We evaluated 133 LeetCode problems drawn from Top Interview 150⁴ (industry interview problems on coding), LeetCode 75⁵ (foundational data structures and algorithms), and Top 100 Liked⁶ (developer community-rated favorites). From these lists, we sampled 41 Easy, 78 Medium, and 14 Hard problems, covering the three difficulty tiers. The sample spans topic categories common in software interviews, such as Arrays/Strings (35%), Trees/Graphs (25%), Dynamic Programming (20%), Sorting/Searching (12%), and Mathematics (8%), with representative tasks such as Two Sum (array hash map), Course Schedule (graph reachability), Coin Change (dynamic programming), and Merge Intervals (interval scheduling). We exclude problems that require visual inputs (e.g., interpreting images) or external interfaces (e.g., issuing HTTP requests to remote services or connecting to external databases).

In this study, our main contributions are as follows:

- (1) Comparing single-agent and multi-agent LLM systems for code generation on 133 programming problems.
- (2) Evaluating success and code quality under a reproducible protocol.
- (3) Analyzing failure patterns and explaining when each system breaks down.
- (4) Releasing the experiment steps and datasets for replication and extension.

The study results show that the multi-agent system produces limited code-quality gains, but incurs high orchestration overhead under fixed settings. Our findings imply that, within the LeetCode-style algorithmic scope we study, some coding problems are better served by the single-agent design, whereas others benefit from the multi-agent orchestration. We distill these measured trade-offs and failure analyses into decision support for practitioners evaluating similar AI coding tools and services. Both orchestration variants use tuned prompts and workflow configurations, so the observed differences arise from the orchestration choices rather than from under-specified agents.

³[www.leetcode.com](https://leetcode.com)

⁴<https://leetcode.com/studyplan/top-interview-150>

⁵<https://leetcode.com/studyplan/leetcode-75>

⁶<https://leetcode.com/studyplan/top-100-liked>

The remainder of this paper is organized as follows: Section 2 provides relevant research on LLM-based code generation and multi-agent systems. Section 3 describes our experimental design and implementation following Wohlin’s guidelines. Section 4 presents our findings, followed by discussion in Section 6. Section 7 addresses threats to validity, and Section 8 concludes.

2 Related Work

Large Language Models can generate functional code across diverse tasks. Prior studies, such as [2, 10], established single-agent approaches on program synthesis and competitive programming tasks. A common setup is single-step prompting from a natural language description to code. This approach works on routine tasks but struggles when problems require decomposition and planning. Most evaluations compare different models or training, and few hold the model constant and compare orchestration designs on the same problems.

Multi-agent approaches [12] instantiate role-specific, LLM-driven agents and coordinate them through a controller that routes messages via patterns such as chain-of-thought, tree-of-thought, or other orchestration flows before merging outputs [14]. Agent roles and executions are designed by developers rather than inferred by AI models. This design can support task decomposition and cross-checking of agent outputs, but it also adds coordination cost, larger contexts, and the risk that early mistakes propagate across agent hand-offs or that agents overfit to specific problem types [14]. Frameworks such as AutoGen [16], ChatDev [11], MetaGPT [7], and CAMEL [9] illustrate these design patterns in software development tasks. However, it remains unclear for which problems multi-agent coordination is beneficial and at what cost relative to single-agent approaches. We address this gap by measuring the acceptance rate of generated solutions, code-quality metrics, and orchestration overhead on 133 diverse problems and by analyzing failure cases to surface the task complexity that favors each orchestration.

Still, evidence from controlled comparisons between single and multiple AI agents is limited. Reports that vary in agent orchestration on the same software tasks, while also presenting code quality and coordination-related issues, are uncommon. Benchmarks with objective pass/fail tests, such as HumanEval and LeetCode acceptance testing, enable controlled evaluation [4]. LeetCode also provides an external human acceptance rate that helps relate results to practice. Motivated by these gaps, our study compares single-agent and multi-agent orchestration under a fixed model on the same LeetCode problems, and reports acceptance rate, code quality, orchestration overhead, and failure patterns alongside the LeetCode human baseline.

3 Research Methodology

We follow Wohlin et al.’s guidelines for controlled experiments [15]. Fig. 1 shows the three phases: Scoping, Planning, and Operation. Analysis and interpretation are presented in Section 4.

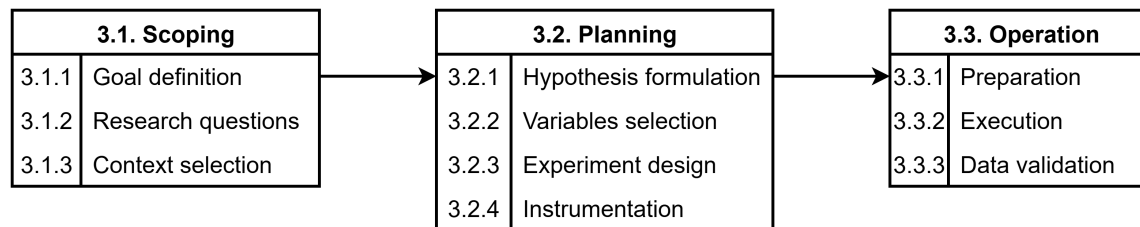


Fig. 1. Overview of the experiment process following Wohlin et al. [15].

3.1 Scoping

The scoping step defines the foundation of our experiment using the Goal-Question-Metric (GQM) template for goal definition [15].

3.1.1 Goal definition. Our goal is to analyze single-agent and multi-agent LLM systems for automated code generation with respect to task success, code quality, and generation time efficiency from the point of view of software engineering practitioners.

3.1.2 Research questions. We formulate three research questions:

RQ1: How do single-agent and multi-agent LLM systems compare in solution acceptance rate for programming problems?

RQ2: What differences exist in code quality metrics (complexity, lines of code) between solutions generated by single-agent and multi-agent systems?

RQ3: What is the orchestration impact (generation time, token usage, and API calls) of adopting multi-agent versus single-agent LLM systems?

3.1.3 Context selection. The selection of appropriate evaluation problems is critical for establishing the validity and generalizability of our findings. We chose LeetCode and defined inclusion and exclusion criteria. **Inclusion:** self-contained, text-only problems solvable from the natural-language prompt with standard input and output. **Exclusion:** any problem that depends on visual inputs (figures or diagrams), interactive or GUI elements, or external resources such as web APIs, databases, files, or linked repositories. As a result, 133 LeetCode problems were selected in three difficulty distributions (e.g., Easy (41), Medium (78), Hard (14)). These criteria bound our conclusions to self-contained, text-only algorithmic programming tasks; multi-modal or interactive problems fall outside the scope of this study. LeetCode problems provide several advantages for a controlled and reproducible experiment:

- *Objective evaluation criteria:* LeetCode pass/fail test case outcomes eliminate subjective bias while verifying a submitted solution for a problem.
- *Standardized difficulty classification:* Well-established Easy/Medium/Hard categories enable analysis across complexity levels.
- *Reproducible test environments:* LeetCode server and infrastructure for automated test execution secure the experiment’s reliability.
- *Human baseline:* Human acceptance rates (55.99% on average ⁷) provide a comparison baseline.

To motivate the selection of LeetCode, we first clarify LeetCode problem structure (see Fig. 2), and then explain their relevance to software engineering and industry, and the fit for multi-agent orchestration in the following paragraphs.

1. What is LeetCode problem – Two Sum Example. LeetCode problems are self-contained programming tasks published with a consistent layout. Each problem page presents (i) a natural-language description of the required functionality, (ii) illustrative input – output examples, and (iii) explicit constraints that delimit input sizes and edge cases. Fig. 2 illustrates a typical LeetCode task using *Two Sum* as the example.

The LeetCode platform provides language-specific function signatures as starter code rather than full solutions. User submissions are evaluated against predefined, hidden automated tests, and the platform reports whether all cases passed, along with runtime and memory statistics.

⁷www.leetcode.com/problemset

LeetCode problem structure	Example problem: Two sum
1. Problem description <Natural language specification>	Given an array of integers and a target sum, return the indices of two numbers that add up to the target.
2. Input/Output examples <Sample test cases with expected results>	Input: nums = [2,7,11,15], target = 9 Output: [0,1] (because nums[0] + nums[1] = 9)
3. Constraints <Input size limits and edge cases>	Constraints: $2 \leq \text{nums.length} \leq 10^4$ Only one valid answer exists.
4. Solution code <Implementation in chosen language>	<pre> from typing import List class Solution: def twoSum(self, nums: List[int], target: int) -> List[int]: num_to_index = {} for i, num in enumerate(nums): complement = target - num if complement in num_to_index: return [num_to_index[complement], i] num_to_index[num] = i </pre> Python3
5. Automated testing <Hidden test cases validate correctness>	
6. Acceptance results <Accepted/Wrong Answer/Others>	

Fig. 2. LeetCode problem structure illustrated with Two Sum.

For *Two Sum*, the problem asks for the indices of two numbers in an array whose sum equals a target value. For example, when `nums = [2, 7, 11, 15]` and `target = 9`, the correct output is `[0, 1]` because `2 + 7 = 9`. Constraints restrict the array length to the range `[2, 104]` and guarantee exactly one valid answer. Editorial solutions and community discussions become available after submission, supporting cross-checking of generated code.

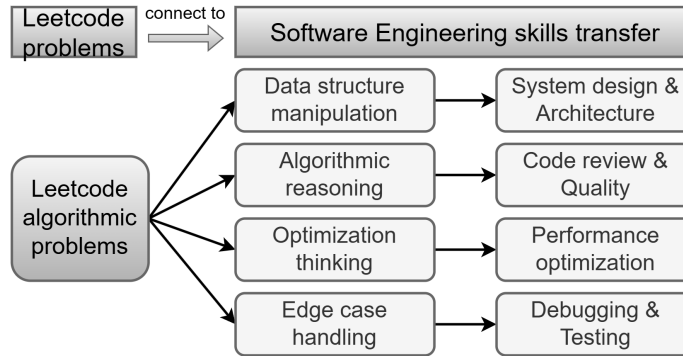


Fig. 3. Connection between LeetCode problems and software engineering skills.

2. Software engineering relevance. Fig. 3 illustrates how LeetCode practice develops data-structure fluency, algorithmic reasoning, optimization thinking, and edge-case handling, which transfer to software engineering skills, such as system design, code review, performance optimization, and debugging and testing.

For example, architects working with hash maps and graphs inform cache keys, indexing, and service topology in system design. LeetCode algorithmic reasoning about invariants and pre-/post-conditions sharpens code reviews by exposing hidden errors and unsafe state changes. Tuning dynamic programming and greedy solutions trains trade-off analysis (precompute vs compute-on-demand) used in software optimization. While exercising adversarial inputs and boundary constraints in LeetCode problems, developers can implement more robust tests for error handling. Together, these skill transfers make LeetCode-style problems a practical and broadly applicable foundation for studying automated programming support.

3. Industry relevance. Industry practice also motivates this dataset choice. Firstly, many technology firms (e.g., Google, Amazon, and Meta) center their technical interview process on algorithmic and data-structure problems, often of LeetCode Medium to Hard difficulty [3]. Secondly, competitive programming performance appears predictive of job readiness: a recent study [1] models how Codeforces participants’ competitive programming metrics correlate with their suitability for various levels of software engineering roles [1]. Thirdly, LeetCode is widely adopted in educational and professional contexts, with the platform reporting 26.3 million monthly visitors in 2025⁸, underscoring its prevalence in both preparation and training for algorithmic problem-solving.

4. Fit for multi-agent orchestration. A useful testbed for multi-agent code generation should offer an objective oracle, a task structure that aligns with analysis–design–implementation–verification, and reproducible inputs without external dependencies. LeetCode meets these needs: problems are self-contained and text-only, the platform issues deterministic pass/fail verdicts, and typical solutions progress from understanding the prompt to selecting a strategy, implementing code, and validating the result. We map these stages to four agents (Fig. 4): *Problem Analyzer* (extracts requirements and key constraints), *Solution Designer* (outlines the algorithmic strategy), *Code Executor* (implements the solution), and *Solution Verifier* (checks examples and flags fixes). This minimal workflow covers parse → plan → code → check while keeping coordination depth short enough to measure efficiency impacts. We do not claim that the selected roles are optimal; rather, they provide a transparent alignment between problem-solving stages and agent responsibilities so that we can attribute effects to orchestration differences. When a task is simple, agents produce concise outputs, but the workflow shows explicit results of each step, allowing us to observe when coordination adds value.

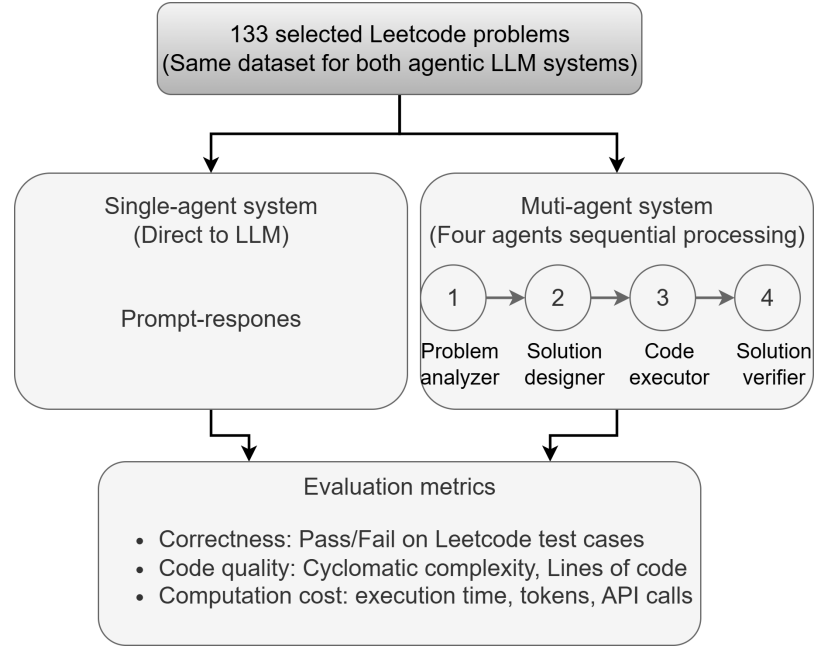


Fig. 4. An overview of experiment design.

⁸<https://leetcode.com/problemset/algorithms>

Fig. 4 shows how we apply this comparison. The top box is the same set of selected LeetCode problems. The left path is the single-agent prompt–response system. The right path is the multi-agent chain in the sequence Analyzer → Designer → Executor → Verifier under a coordinator. Both paths terminate in the same evaluation metrics: pass/fail verdicts from the LeetCode evaluation service, code-quality measures (cyclomatic complexity and lines of code), and efficiency indicators (generation time, with tokens and API calls reported as resource footprint). In this setup, only the orchestration logic differs; prompts are tailored to the respective steps, but the underlying model and problem set remain identical, letting us attribute differences to the coordination strategy.

3.2 Planning

Planning covers hypothesis formulation, variable selection, experiment design, instrumentation, and validity evaluation.

3.2.1 *Hypothesis formulation.* We formulate the following hypotheses for each research question:

For RQ1 (The acceptance of the generated solution):

- H_{01} : There is no difference in solution acceptance rate between single-agent and multi-agent systems.
- H_{11} : Multi-agent systems achieve a higher solution acceptance rate than single-agent systems.

For RQ2 (The quality of generated code):

- H_{02a} : The mean cyclomatic complexity is equal between single-agent and multi-agent systems.
- H_{12a} : The mean cyclomatic complexity differs between single-agent and multi-agent systems.
- H_{02b} : The mean lines of code are equal between single-agent and multi-agent systems.
- H_{12b} : The mean lines of code differ between single-agent and multi-agent systems.

For RQ3 (Computational latency):

- H_{03} : There is no difference in latency between single-agent and multi-agent systems.
- H_{13} : Multi-agent systems require higher latency than single-agent systems.

3.2.2 *Variables selection.* Table 1 presents the variables used in our experimental design. The first author identified and operationalized these variables based on the research questions and established software engineering metrics.

Table 1. Selected variables

Variable Type	Variable name	Description
Independent	System Type	Single-agent vs Multi-agent (categorical)
Dependent	Acceptance rate	Percentage of solutions marked “Accepted” by LeetCode (ratio)
	Lines of Code	Total executable lines in generated solution (ratio)
	Cyclomatic Complexity	McCabe complexity measure (ratio)
	Generation Time	Time required to generate solution (ratio)
	Token Usage	Input and output tokens consumed (ratio)
	API Calls	Number of LLM requests per problem (ratio)
Controlled	Underlying LLM	GPT-4o for both systems
	Model Parameters	Temperature = 0 for reproducibility
	Problem Set	Identical 133 problems for both systems
	Evaluation Criteria	Same LeetCode test suite

Each variable in Table 1 was carefully defined to ensure measurability and reproducibility. Acceptance Rate records the proportion of submissions that LeetCode labels “Accepted.” Lines of Code excludes comments and blank lines using a language-aware parser. Cyclomatic Complexity is computed on the generated abstract syntax tree via the standard McCabe algorithm. Generation Time measures the time from issuing the first model call until the final code artifact is produced. Token Usage comes directly from the API response metadata, and API Calls counts the number of LLM requests per problem.

3.2.3 Experiment design. To enable paired statistical comparisons, both systems solve the same set of problems. Holding the problem set constant isolates orchestration as the only varying factor and increases power for detecting differences between the two approaches.

Single-Agent System: GPT-4o receives the entire problem statement plus a coding instruction prompt and returns a complete solution in one request. No intermediate reasoning is exposed; any self-checking must occur implicitly within that single call.

Multi-Agent System: GPT-4o is invoked four times with role-specific prompts. The Problem Analyzer summarizes requirements and constraints; the Solution Designer proposes an algorithm with complexity notes; the Code Executor produces runnable code following that plan; and the Solution Verifier inspects the code, replays examples, and can trigger a regeneration if issues are found. The coordinator streams the conversation history between these stages.

3.2.4 Instrumentation. To ensure reproducibility and controlled evaluation, we implemented scripts and interfaces in GO to automate problem downloading, GPT-4o interactions, submissions of generated code, and extraction of LeetCode testing verdicts. The complete implementation is available as an open-source repository⁹ that enables replication of our experiment process.

Our implementation consists of several integrated components. The problem acquisition (download.go) module automatically downloads problem descriptions, constraints, and test cases from the three selected LeetCode study plans using the LeetCode API. The solution generation module implements both single-agent (prompt.go) and multi-agent (multiagent.go) approaches, managing API calls to GPT-4o with appropriate prompts and coordination logic for the multi-agent system.

The evaluation component automatically submits generated solutions to LeetCode’s testing platform and captures acceptance rates, runtime performance, and error messages. Our metrics extraction component analyzes generated code to compute cyclomatic complexity using established algorithms, counts lines of code excluding comments and blank lines, and measures token usage and generation time.

The data module stores experiment data in structured JSON format, maintains detailed logs of API interactions and system responses, and provides automated scripts for statistical analysis and result compilation. Quality assurance mechanisms include validation of problem downloads against expected formats, verification of solution submissions and result capture, and consistency checks across all generated metrics.

Our goal is repeatability rather than bitwise determinism. We used the same model (GPT-4o) and problem set, and we logged all inputs/outputs for re-execution. Because GPT-4o APIs are not strictly deterministic, we re-ran the full protocol three independent times and report means with 95% confidence intervals across replications. Minor run-to-run variation occurred on a small subset of problems, but the conclusions are unchanged. We release code, prompts, logs, and outputs to enable independent re-runs.

⁹<https://github.com/Liang-Y-Yu/LeetCodeEval.git>

3.2.5 *Statistical analysis.* We employed different statistical tests appropriate for each type of data and research question. For acceptance rate comparison (RQ1), we used Fisher’s exact test to compare the binary outcomes (pass/fail) between systems. Fisher’s exact test is preferred over chi-square tests for categorical data with small expected frequencies, as it provides exact p-values regardless of sample size. The test evaluates the null hypothesis that both systems have identical acceptance rates by calculating the probability of observing the actual contingency table (or more extreme) under this assumption.

For code quality metrics (RQ2), we used two-sample t-tests to compare continuous variables (lines of code, cyclomatic complexity) between systems. We verified normality assumptions using Shapiro-Wilk tests and reported both statistical significance (p-values) and practical significance (Cohen’s d effect sizes). For the latency analysis (RQ3), we used descriptive statistics and ratio comparisons, as the differences are substantial and clearly meaningful without requiring significance testing. All statistical analyses were conducted using established software packages with $\alpha = 0.05$ as the significance threshold.

3.3 Operation

Following Wohlin’s guidelines [15], the operation phase executes the plan: prepare the controlled setup, run both systems on the same tasks, and validate outputs.

3.3.1 *Preparation.* We prepared a controlled environment to ensure the reliability and reproducibility of the experiments. This environment ran Ubuntu 20.04 LTS with 64 GB RAM and stable internet connectivity to minimize network-related variability. The preparation phase involved several critical steps, such as configuring API access, accommodating LeetCode submission constraints, and validating the problem dataset.

The first step, API access configuration, required obtaining Azure credentials with sufficient quota to handle the expected token volume (about 597 K across all experiments) and implementing rate limiting to respect API constraints while maintaining consistency. The token estimate came from pilot runs on a full problem set.

LeetCode submissions occasionally failed due to rate limiting and service protections (HTTP 403/5xx). To remain compliant with the platform, we throttled request rates and scheduled runs during off-peak hours. These operational constraints extended the preparation time but did not change the evaluation protocol. We logged all submission outcomes for auditability.

The problem dataset preparation involved downloading and validating LeetCode problems. During the validation, we found four initially selected problems that violated our inclusion and exclusion criteria defined in Section 3.1.3. We replaced them with alternatives from the same study plans and difficulty levels to preserve the difficulty mix.

3.3.2 *Execution.* The experiment was executed in three independent replications during off-peak hours in September 2025, each spanning a 72-hour window to reduce variance from LeetCode and GPT-4o traffic load. Preliminary tests during peak hours showed 15–20% higher response-time variability, motivating the off-peak schedule. In every replication, we ran two sequential phases on the same 133 problems: first, the single-agent system, then the multi-agent system.

We reset the environment between phases by opening a fresh session, clearing conversation history, and reloading the problem prompt and tests. Prompts, model, and rate limiting were identical across replications. We controlled context growth with dynamic truncation that preserved instructions. We applied exponential backoff with capped retries, timeouts for stalled responses, and fallbacks for malformed handoffs in the verification step.

We logged timestamps for each GPT-4o call and recorded per-problem metrics (accept/fail outcome, cyclomatic complexity, lines of code, generation time, token counts, and number of calls). Our analysis aggregates these per-problem metrics across the three replications and reports means with 95% confidence intervals.

3.3.3 Data validation. We implemented multiple validation layers covering both systems to ensure data integrity and experimental validity.

Real-time validation comprised automated checks for API response completeness, automated linters that ensured generated code compiled in Python, and confirmation that LeetCode acknowledged each submission.

Post-execution validation re-ran integrity checks. We verified that both systems processed all 133 problems, reconciled LeetCode submission verdicts with our local logs, and recomputed code metrics to confirm consistency. When we detected cases where the platform returned inconsistent verdicts for identical code, we immediately resubmitted within the same replication to obtain stable results.

We also manually reviewed a uniform random sample of 27 multi-agent conversations (about 20% of the dataset) to verify consistency between logs and outputs. The review confirmed that the implementation matched the intended design, and all planned metrics were collected for every problem.

4 Results

This section presents our findings to answer research questions.

4.1 Results of RQ1: Multi-agent systems do not improve acceptance rate

Our RQ1 examines whether multi-agent orchestration improves the solution acceptance rate on the same 133 problems. The single-agent system remained at 98.50% in every one of the three independent replications with identical inputs, whereas the multi-agent system ranged from 96.99% to 97.74%. We report the per-run outcomes in Table 2 and the average across runs in Table 3. For external context only, the human baseline (55.99%) comes from LeetCode’s webpage (accessed on 7th September 2025).

Table 2. LeetCode acceptance rates by replication on the same 133 problems.

Experiment run	Single-Agent	Accepted/Failed	Multi-Agent	Accepted/Failed
1st run (5th September)	98.50%	131/2	97.74%	130/3
2nd run (6th September)	98.50%	131/2	96.99%	129/4
3rd run (7th September)	98.50%	131/2	96.99%	129/4

Table 3. Acceptance rate averaged over three replications (same 133 problems).

System type	Mean acceptance rate	Standard deviation (percentage points)
Single-Agent	98.50%	0.00
Multi-Agent	97.24%	0.43

For each replication, we applied Fisher’s exact test with a one-sided alternative that the multi-agent pass rate exceeds the single-agent pass rate [5]. For multi-agent, 1st run is $p = 0.815$; 2nd run is $p = 0.893$; and 3rd run is $p = 0.893$.

Averaging across runs, the mean acceptance-rate difference is -1.26 percentage points (multi minus single) with a 95% confidence interval across the three replications of $[-2.35, -0.17]$ percentage points. These results do not show an advantage for multi-agent orchestration in acceptance under a fixed model.

To better understand whether the similar overall acceptance rates mask differences in how each system handles problems of varying complexity, we analyzed performance across LeetCode’s difficulty categories. Table 4 reports the breakdown by LeetCode difficulty averaged over the three replications, and the pattern is consistent in each individual run.

Table 4. Acceptance rates by problem difficulty

Difficulty	Count	Human	Single-Agent	Multi-Agent
Easy	41	58.85%	100.0%	100.0%
Medium	78	56.30%	97.4%	96.2%
Hard	14	45.83%	100.0%	100.0%

The difficulty breakdown shows similar acceptance rates across Easy, Medium, and Hard in our dataset. Across the three replications, both systems solved all Easy and Hard problems, and the Medium category differed by roughly one percentage point in favor of the single-agent system. Because Easy and Hard results are at ceiling and the Hard sample is small (14 tasks), we refrain from generalizing to harder tasks beyond this set. We therefore interpret the difficulty breakdown as showing no observed advantage for the multi-agent system within our dataset, without extending the claim to more complex tasks in general.

Conclusion for RQ1: We fail to reject H_{01} in each replication using a one-sided Fisher test. With three replications on the same problems, the average acceptance favors the single-agent system by about 1.26 percentage points, and we find no evidence that multi-agent systems increase acceptance under a fixed model and prompts.

4.2 Results of RQ2: Multi-agent systems yield small code-quality gains

While multi-agent systems show no advantage in LeetCode acceptance rate, they show measurable but modest improvements in code-quality metrics. Table 5 reports the aggregate means, and Fig. 5–6 visualize per-problem patterns and variability across the three replications.

Table 5. Code-quality metrics comparison

Metric	Single-Agent	Multi-Agent	Change (multi–single)
Lines of Code (mean)	15.24	14.69	-3.6%
Cyclomatic Complexity (mean)	8.33	7.93	-4.8%

Using per-problem means across the three runs, the Multi-Agent system reduces cyclomatic complexity (7.93 vs 8.33, $p < 0.05$, Cohen’s $d = -0.26$) and lines of code (14.69 vs 15.24, $p < 0.05$, Cohen’s $d = -0.09$). Cohen’s negative values indicate that the multi-agent system produces solutions with slightly lower cyclomatic complexity and solutions with fewer lines of code, but the effect size is small, suggesting the practical difference is minimal.

The effect size matches the visual impression in Fig. 5, where most problem pairs sit close together. Fig. 6 shows that run-to-run variability is low for the single-agent system and generally low for the multi-agent system, with a few

Standard Deviation of Lines of Code

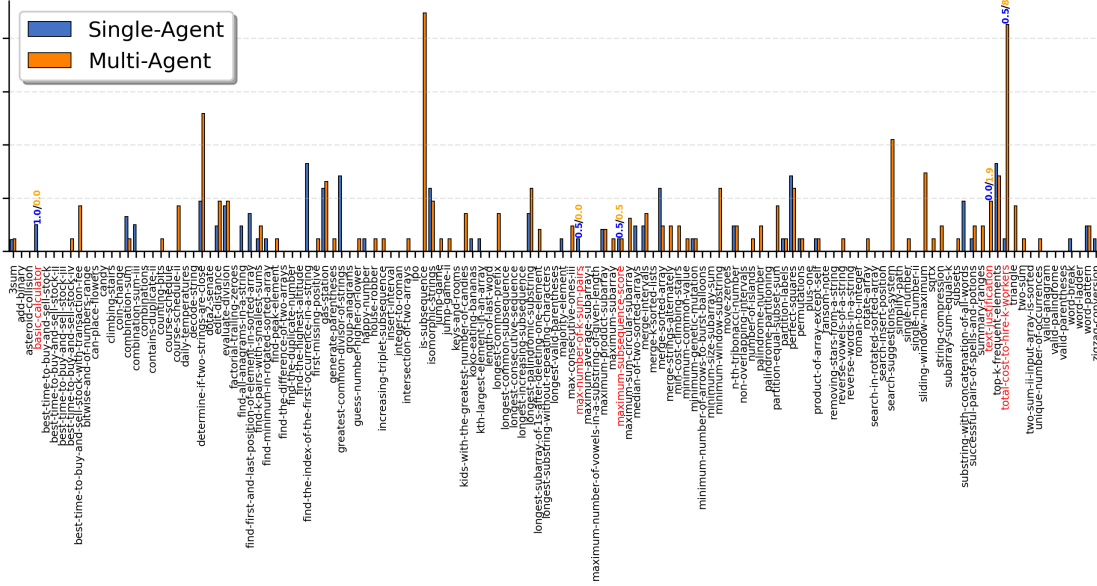


Fig. 6. Per-problem standard deviation of lines of code across the three replications. Variability is small for most problems, with a few visible spikes for the multi-agent system. Those problems highlighted in red were not accepted by the LeetCode submission system.

Table 6. Latency-focused comparison.

Metric	Single-Agent	Multi-Agent
Average generation time (s)	2.1	8.3
Average input tokens	453	3,376
Average output tokens	83	583
API calls per problem	1	4

Instrumentation supports these contributors. Per handoff, we observed delays in routing and queuing overhead in the coordinator. Token use is concentrated in the analysis and design stages (Problem Analyzer ≈ 892 input tokens, Solution Designer $\approx 1,124$, Code Executor ≈ 847 , Solution Verifier ≈ 513), which lengthens processing even when the final code is short. In one specific case (the Basic Calculator problem), the verifier requested a regeneration, adding another full pass through at least one agent.

The overhead analysis also reveals inefficiencies in the current multi-agent design. The Problem Analyzer agent often produces detailed analysis that overlaps with information already available in the problem statement. The Solution Designer agent frequently repeats algorithmic concepts, contributing to token usage without proportional benefit. These inefficiencies suggest potential optimizations that could reduce costs while maintaining quality benefits, and they also hint that relatively short algorithmic tasks may not need a multi-agent system.

Conclusion for RQ3: Based on our latency analysis, we reject the null hypothesis H_{03} and accept the alternative hypothesis H_{13} . Under a fixed model, multi-agent orchestration increases end-to-end generation time by about four times on these tasks, largely due to additional LLM calls and larger contexts rather than the model choice.

5 Implications

This section presents the implications of the study results.

5.1 Failure pattern in different systems

The failure patterns between single-agent and multi-agent systems reveal fundamentally different breakdown mechanisms. Table 7 presents the specific problems where each system failed, along with the failure modes and underlying causes.

Table 7. Failed problems and failure types (unique problem IDs aggregated across three replications)

Problem Name	System	Failure Type	Analysis description
Maximum Subsequence Score	Single-Agent	Wrong Answer	Classic algorithmic logic error in solution implementation
Total Cost to Hire K Workers	Single-Agent	Wrong Answer	Incorrect algorithmic approach, 31 lines
	Multi-Agent	Wrong Answer	Different implementation but same error, 34 lines
Basic Calculator	Multi-Agent	Wrong Answer	Coordination process introduced implementation bugs not caught by the verifier
Substring with Concatenation of All Words	Multi-Agent	Time Limit Exceeded	Less efficient algorithm despite dedicated Solution Designer agent
Text justification	Multi-Agent	Runtime Error	LeetCode test case runtime error

The single-agent system failed on two unique problems with consistent ‘Wrong Answer’ verdicts, representing classic algorithmic logic errors where the system understood the problem but implemented incorrect solution logic. In contrast, the multi-agent system exhibited more diverse failure modes across three unique problems (four failure instances across the replications), including ‘Runtime Errors’ and timeout issues that suggest coordination-related problems.

The most revealing LeetCode problem is “Total Cost to Hire K Workers ¹⁰,” where both systems failed across all three replications but produced different implementations. The multi-agent system introduced an extra verifier pass with fewer lines of code in the final solution, yet still produced the wrong answer, indicating that coordination led to an alternative but equally flawed algorithmic choice.

The “Basic Calculator ¹¹” runtime error particularly highlights how agent handoffs can introduce implementation bugs that would not occur in a single, coherent generation process. This problem also shows the largest per-problem gap in lines of code between two systems in our dataset, with the multi-agent implementation being substantially longer than the single-agent one (see Fig. 5). The co-occurrence of a longer implementation and a runtime failure suggests a coordination-related failure, and the verifier did not catch the introduced defect.

This pattern suggests that multi-agent systems introduce coordination costs that can actually degrade solution quality. The specialized agents may optimize for their individual objectives (analysis thoroughness, design elegance, implementation completeness) without sufficient consideration of the overall solution’s effectiveness.

¹⁰<https://leetcode.com/problems/total-cost-to-hire-k-workers/description/>

¹¹<https://leetcode.com/problems/basic-calculator/description/>

5.2 Code quality and latency

While multi-agent systems produced improvements in code quality metrics, the practical significance of these gains warrants careful examination. The 4.8% reduction in cyclomatic complexity and 3.6% reduction in lines of code represent modest improvements that may not justify the longer generation time.

Our analysis of the complexity distribution reveals that both systems primarily generate low-complexity solutions. The multi-agent system’s advantage appears most noticeable in this low-complexity category, where the Solution Designer agent can identify simpler algorithmic approaches. However, for higher complexity problems that require sophisticated algorithmic insights, the coordination overhead may hinder performance.

5.3 Latency – generation time

We treat end-to-end generation time as the efficiency metric. Table 6 already shows the $\sim 4\times$ slowdown for the multi-agent system; here we emphasize the structural contributors. Sequential orchestration requires at least four model calls per problem versus one for the single-agent baseline, adding three network round-trip and startup costs even when prompts are short. Each call also carries a larger prompt (mean input tokens 3,376 versus 453; mean output tokens 583 versus 83), so every stage incurs longer prefill and decode times. Verifier-triggered regenerations (about 12% of problems) and small coordination overheads at hand-offs further widen the gap.

Prompt design can narrow but not remove the gap. Shorter handoff schemas, selective context (only fields needed by the next agent), and gating the verifier can reduce token counts and avoid extra passes. In a sequential four-agent chain, there is a structural latency floor, which requires at least four model invocations and three hand-offs, even with brief messages. We therefore report generation time as the outcome of interest and include token usage and LLM calls as explanatory footprint measures.

5.4 Implications for system design and deployment

Our findings suggest that adding more agents may reach a point where coordination costs outweigh specialization benefits. Within the short, LeetCode-style problems we studied, single-agent systems appear to offer the best balance of quality and latency for routine code generation tasks (e.g., scripting or small refactorings).

Our failure pattern analysis suggests that multi-agent systems may be more appropriate for tasks where additional verification passes are valuable, for example, problems that demand explicit checking against numerous edge cases, rather than for initial generation of short snippets. The diverse failure modes we observed indicate that multi-agent coordination introduces new categories of errors that do not exist in single-agent systems.

6 Discussion

We discuss study results and their impacts in this section.

6.1 Error propagation in sequential agent chains

When agents operate in a chain-of-thought pattern where each agent builds upon the previous agent’s output, a single error in the middle of the chain can cascade through all subsequent agents. Potentially, this kind of error may lead to a completely incorrect final solution regardless of how well the later agents perform their individual tasks.

Consider our four-agent architecture: Problem Analyzer → Solution Designer → Code Executor → Solution Verifier. If the Problem Analyzer misunderstands the problem requirements, this misunderstanding propagates to the Solution

Designer, who creates a design based on incorrect assumptions. The Code Executor then implements this flawed design, and the Solution Verifier validates against the wrong criteria. In this scenario, the entire system’s quality is bounded by the Problem Analyzer’s initial error, regardless of how sophisticated the subsequent agents are.

Thus, error propagation could be a scalability problem with sequential multi-agent systems for code generation purposes. The Basic Calculator failure in Table 7 illustrates this pattern: a misinterpretation in the Problem Analyzer led the Solution Designer to adopt an incorrect stack strategy, which the Code Executor implemented and the Verifier accepted, yielding the same runtime error across replications. When agents operate in a sequential chain where each agent builds upon the previous agent’s output, the probability of overall system success decreases exponentially with chain length. If each agent has a probability p of correctly processing its input, then a chain of n agents has an expected success probability of p^n . As the chain length approaches infinity, this probability approaches zero: $\lim_{n \rightarrow \infty} p^n = 0$ for any $p < 1$. This simple probability reasoning assumes independent pass/fail and binary correctness; real agent interactions can be more nuanced because outputs may be ambiguous, partial, or open to multiple valid interpretations (e.g., different sorting algorithms with similar behavior). Nonetheless, even highly reliable individual agents (e.g., $p = 0.95$) will see overall reliability decline as chains grow longer: a four-agent chain would have approximately 81% success ($0.95^4 \approx 0.81$), while a ten-agent chain would drop to about 60%.

These dynamics raise design questions about where to place confirmation checkpoints that break error propagation and how to implement them effectively. Candidates include a human-in-the-loop review or an out-of-chain validator agent that consults the original prompt, executes local tests, and either passes or blocks the handoff.

Additionally, multi-agent systems do not just redistribute existing failure modes. They might create new categories of failures. Runtime errors from coordination bugs, timeout failures from inefficient agent communication, and verification failures from misaligned agent objectives represent failure modes that cannot occur in single-agent systems. Future research should investigate coordination mechanisms that break error propagation chains, develop adaptive systems that can detect and recover from coordination failures, and establish theoretical bounds on when multi-agent approaches can outperform single-agent systems.

6.2 When to use single-agent and multi-agent for maximized value

Our findings reveal a trade-off pattern between the breadth of applicability and the chance of getting expected outputs in single-agent versus multi-agent systems. As illustrated in Fig. 7, a hypothesis is that single-agent systems exhibit a broad benefit distribution, delivering consistent results across a wide range of software engineering tasks, while multi-agent systems demonstrate a narrow but peaked benefit distribution, achieving higher chances of expected outputs within a limited scope of applications.

Within programming tasks similar to our datasets (e.g., algorithmic problems with objective tests), the single-agent delivered stable results at lower latency. For routine algorithmic coding in this regime, a single agent is a reasonable default. However, for large projects, interactive human-in-the-loop tasks, or UI/UX work, further studies are required.

Multi-agent systems operate differently, achieving peak chances of expected outputs within a narrow range of applications. While our findings showed improvements (4.8% complexity reduction, 3.6% fewer lines), these gains represent meaningful advances for specific problem types requiring extensive decomposition, design validation, and iterative refinement. The coordination overhead that reduces multi-agent effectiveness in routine tasks becomes valuable when problems align with their specialized capabilities. However, when problems fall outside this narrow beneficial range, the coordination complexity decreases rather than increases the likelihood of achieving expected outputs. Multi-agent systems may offer a *narrow but peaked* benefit: small average gains in code quality (-4.8% cyclomatic

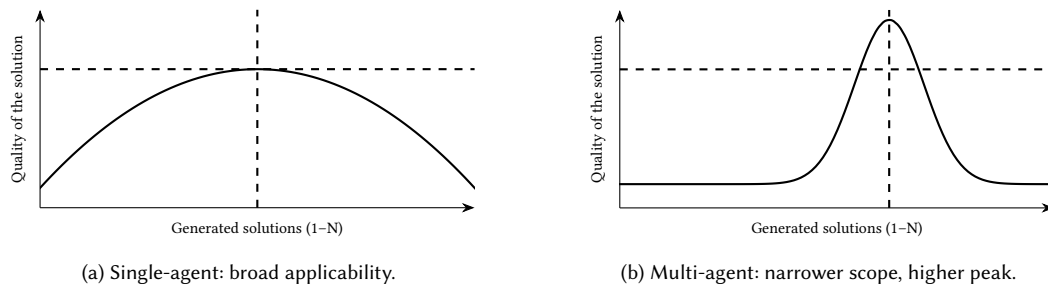


Fig. 7. The single-agent is hypothesized to yield a broad benefit across tasks, whereas the multi-agent concentrates a higher probability of achieving the desired output within a narrower task region.

complexity, -3.6% lines of code) at the cost of higher latency. We read this as a hypothesis supported by our observations on LeetCode-style tasks that demand decomposition or explicit checking, not as a general claim. Our dataset contains algorithmic problems on code generation, and we did not study large-scale projects and non-coding tasks (e.g., text analysis, human-machine interactions, and data processing). For those tasks not covered by this study, the balance could differ, and the coordination overhead might either help or hurt. We therefore present the *narrow but peaked* pattern as a theory for discussion and a target for future tests across other tasks and agent designs.

This distribution pattern explains why aggregate statistics may underestimate multi-agent advantages in their optimal domain while overestimating their general utility. Our analysis of those LeetCode problems with ‘Wrong Answer’ feedback supports the idea that multi-agent systems failed through different mechanisms than single-agent systems, suggesting they operate in a distinct regime for producing expected outputs. When agents in a chain work as expected, multi-agent systems can deliver similar results as a single agent does. As such, multi-agent systems should be used when tasks require decomposition, planning, or verification that a single agent cannot handle well.

We expect future work to investigate intelligent orchestrations that can route problems to single-agent or multi-agent systems based on task features and risk. Such routing can improve the acceptance rate of AI-generated outcomes and reduce the latency across a broad range of software engineering tasks.

7 Threats to Validity

We followed Wohlin et al.’s guidelines [15] to address four types of validity threats.

7.1 Internal Validity

Our aim was to attribute differences to orchestration rather than hidden factors. We fixed the model and prompts, used the same 133 tasks, and executed three off-peak replications to reduce volatility. However, GPT-4o APIs are not strictly deterministic, so point estimates can drift across weeks; we therefore report means and 95% confidence intervals and avoid claims that hinge on small p -value swings.

Run order is a possible concern because the single-agent phase preceded the multi-agent phase in each replication. We opened fresh sessions, cleared history, and reloaded prompts to block cross-run leakage, yet timing and background load can still vary and affect latency. Aggregation over three replications mitigates this risk but does not remove it.

The task sample can also bias results within our scope. We drew from three LeetCode study plans under the inclusion criteria for self-contained, text-only problems. This set may under-represent tasks where decomposition helps most, so the findings apply to this regime and not to all programming tasks.

Design choices are another source of threat. Our four-role chain is a defensible mapping of parse $\rightarrow$ plan $\rightarrow$ code $\rightarrow$ check, but other role sets, tighter handoff schemas, or verifier gating could change token use, generation time, and even quality.

7.2 Construct Validity

Acceptance rate is operationalized as LeetCode *Accepted* on the public judge. This maps to functional correctness on the LeetCode platform but does not cover all industrial test scenarios or non-functional requirements. Code quality is proxied by cyclomatic complexity and lines of code. These proxies capture structural simplicity and size, not readability, naming, architecture, or security. We report both metrics to avoid reliance on a single proxy and to enable replication.

Latency is the end-to-end *generation time* while accessing the GPT-4o APIs. This is a system-latency measure from the first request to the final submission. Token counts and API calls are reported as resource trace, not as monetary or energy cost. We fix the model (GPT-4o) to isolate orchestration effects. Therefore, our findings speak to orchestration at this capability level. A stronger or weaker model could shift absolute levels while preserving or altering the observed gaps; testing that is outside our scope.

The task context is self-contained algorithmic problems with objective oracles. This context fits RQ1–RQ3 for code generation on well-specified tasks, but under-represents large-scale projects or interactive AI assistant tasks. Additionally, LeetCode’s published mean acceptance rate for human-submitted solutions is used for external context only.

7.3 External Validity

A threat to our findings lies in the rapid evolution of LLM capabilities. We employed GPT-4o on Azure API version 2024-10-21, but the AI landscape changes dramatically every few months. Future models may possess fundamentally different reasoning capabilities that could shift the single-agent versus multi-agent balance.

Another concern is platform drift: LeetCode’s test cases and problem statements can change over time. To mitigate this, we recorded problem identifiers and our tooling downloaded the statements. We release the problem list and timestamps so future studies can either reuse the problem set or re-evaluate it. Even with these steps, accepted results should be read as time-bound to the LeetCode state during our runs. Generalizability is also limited by context. Our tasks are self-contained algorithmic problems, and they do not represent other non-coding problem types.

7.4 Conclusion Validity

We matched statistical tests to data types, using a one-sided Fisher’s exact test for RQ1 (multi-agent $>$ single-agent) and two-sided t-tests for the continuous metrics in RQ2, with $\alpha = 0.05$. Power is adequate for differences at the scale observed across 133 problems, but the small number of failures limits failure-pattern analysis. Assumption checks (e.g., Shapiro–Wilk) preceded t-tests for code-quality metrics. Ratios and descriptive statistics summarize cost and time because the claim does not require hypothesis tests. Results are consistent across problem difficulties, which supports the stability of our conclusions.

8 Conclusion

We ran a controlled experiment on 133 LeetCode problems with a fixed model to compare a single-agent system against a four-agent system. We repeated the experiment three times on the same problem set to reduce run-to-run noise and to report stable results.

For **RQ1**, we found no acceptance rate advantage for the multi-agent orchestration. The average acceptance rate was 98.50% for the single-agent system and 97.24% for the multi-agent system, and one-sided tests on each replication failed to show an improvement for the multi-agent system. For **RQ2**, the multi-agent system produced small code-quality gains. Cyclomatic complexity dropped by about 4.8% and lines of code by about 3.6%, with small effect sizes. Figures with per-problem means and standard deviations show that most problems exhibit tight pairs with modest differences. For **RQ3**, the multi-agent system incurred about four times higher end-to-end generation time. The main drivers were additional API turns and larger contexts from agent handoffs, with occasional verifier-triggered regeneration.

Taken together, these results support two guidelines: (i) use the single-agent system as the default for algorithmic programming tasks where fast responses matter; (ii) use multi-agent orchestration selectively when decomposition, explicit review, or strict quality targets are required and the added latency can be tolerated.

This study also points to design opportunities. Reduce agent turns, constrain analysis verbosity, summarize state between agents, and route only the harder problems to the multi-agent design. Human or auxiliary checkpoints outside the main chain may help break error propagation.

Our conclusions are bound by the setting. The tasks are algorithmic, the code artifacts are small, and we used one fixed model. Other task types and larger codebases may show different trade-offs. We release code and data to support replication and extension. Future work includes adaptive orchestration that selects between single- and multi-agent designs based on task features, checkpoint strategies to limit error propagation, and studies on larger software tasks beyond algorithmic problems.

Acknowledgments

We acknowledge support from the Knowledge Foundation through the S.E.R.T. Research Profile Project (research profile grant 20180010).

Data and Code Availability

All code, prompts, and raw results are available at <https://github.com/Liang-Y-Yu/LeetCodeEval>. The repository includes a README with instructions to reproduce all tables and figures.

References

- [1] Md Imranur Rahman Akib, Fathima Binthe Muhammed, Umit Saha, Md Fazlul Karim Patwary, Mehrin Anannya, Md Alomgeer Hussein, and Md Biplob Hosen. 2025. From Code to Career: Assessing Competitive Programmers for Industry Placement. *arXiv preprint arXiv:2508.00772* (2025).
- [2] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021).
- [3] Mahnaz Behroozi, Shivani Shirolkar, Titus Barik, and Chris Parnin. 2020. Does stress impact technical interview performance?. In *Proceedings of the 28th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering*. 481–492.
- [4] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [5] Ronald A. Fisher. 1922. On the interpretation of χ^2 from contingency tables, and the calculation of P. *Journal of the Royal Statistical Society* 85, 1 (1922), 87–94.
- [6] GitHub. 2021. GitHub Copilot. <https://github.com/features/copilot>
- [7] Sirui Hong, Xianwu Zheng, Jonathan Chen, Yuheng Cheng, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, et al. 2023. MetaGPT: Meta programming for a multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352* (2023).
- [8] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 1–79.
- [9] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. CAMEL: Communicative agents for "mind" exploration of large scale language model society. In *Advances in Neural Information Processing Systems*, Vol. 36. 51991–52008.

- [10] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. Competition-level code generation with alphacode. *Science* 378, 6624 (2022), 1092–1097.
- [11] Chen Qian, Xin Cong, Cheng Yang, Weize Chen, Yusheng Su, Juyuan Xu, Zhiyuan Liu, and Maosong Sun. 2023. Communicative agents for software development. *arXiv preprint arXiv:2307.07924* (2023).
- [12] Peter Stone and Manuela Veloso. 2000. Multiagent systems: a survey from a machine learning perspective. *Autonomous Robots* 8, 3 (2000), 345–383.
- [13] Priyan Vaithilingam, Tianyi Zhang, and Elena L Glassman. 2022. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts* (2022).
- [14] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. 2023. A survey on large language model based autonomous agents. *arXiv preprint arXiv:2308.11432* (2023).
- [15] Claes Wohlin, Per Runeson, Martin Höst, Magnus C Ohlsson, Björn Regnell, and Anders Wesslén. 2012. *Experimentation in software engineering*. Springer Science & Business Media.
- [16] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. 2023. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155* (2023).